

第9章 并发与异常

建议学时:6-8 小时 | 难度:★★★★★ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握异常体系:Error / Exception / RuntimeException 的分层与意义
- 理解 checked 与 unchecked 异常的区别,掌握声明与处理的规则
- 掌握 try-catch-finally、多 catch、自定义异常的最佳实践
- 掌握线程的创建方式与生命周期
- 掌握 synchronized 与原子类,理解"竞态"与"可见性"
- 掌握 java.util.concurrent:线程池、并发容器
- 理解死锁成因,建立并发安全的工程习惯

💡 从 C 到 Java:错误处理与并发的范式革命

异常是对 errno 的全面升级。 C 的错误处理靠返回值 + errno:每个函数都要检查返回值,忘了检查就静默出错;错误信息靠 perror 打印。Java 用异常——错误发生时"抛出"一个对象,沿调用栈向上找处理者。强制处理的 checked 异常,是编译器逼你把错误处理写进代码;不强制处理的 unchecked 异常,是编程错误(空指针、下标越界)。C 程序员最需要适应的:不是"怎么抛",而是"哪些必须处理、哪些不该处理"。

并发:从 pthread 到语言内建。 C 的线程是 pthread 库的第三方体验:pthread_create/mutex_lock 全是函数;Java 里 Thread / synchronized 是语言与标准库的一等公民。但 并发难题——竞态、可见性、死锁——一模一样,C 里怎么踩坑,Java 里换个马甲继续踩。核心思维不变:共享可变状态是万恶之源,要么锁,要么别共享,要么用原子操作。

线程池是生产标配。 C 程序员裸建线程(像第8章的裸 Thread);生产 Java 用 Executors/ThreadPoolExecutor 管理线程生命周期——池化复用、上限可控、队列缓冲。这是"资源管理"思维在并发领域的体现,和第5章"内存由 GC 管"同源:别亲手持有稀缺资源,交给专业管家。

学习顺序:先异常(\$9.1-9.3,独立主题),再线程基础(\$9.4),同步(\$9.5),并发工具(\$9.6),最后死锁与纪律(\$9.7)。

📦 前置知识自查

- C 的 errno/返回码错误处理;pthread 线程与互斥锁的基本概念
- 第8章:多客户端线程池已"预装"过 Executors 的用法
- 第4章:lambda(线程任务用 Runnable);第6章:对象与封装

🚀 本章学习路线

1. **第一阶段(约 2 小时):**\$9.1-9.3 异常体系,把"返回值检查"思维翻译成"异常处理"思维
2. **第二阶段(约 2 小时):**\$9.4-9.5 线程与同步,亲手制造一次竞态再修好
3. **第三阶段(约 2 小时):**\$9.6-9.7 并发容器/线程池/死锁,完成章末实验
4. **验收标准:**能画出异常体系图并说出 checked/unchecked 规则;能解释竞态与可见性;能写出线程池并发统计

本章知识导图

- §9.1 异常体系:Throwable / Error / Exception
- §9.2 checked 与 unchecked:声明、处理、取舍
- §9.3 try-catch-finally 与自定义异常
- §9.4 线程:创建与生命周期
- §9.5 同步:synchronized / 原子类 / volatile
- §9.6 java.util.concurrent:线程池与并发容器
- §9.7 死锁与并发纪律

§9.1 异常体系总览

9.1.1 Throwable 家族树

类型	代表	含义	C 对应	必须处理?
Error	OutOfMemoryError、 StackOverflowError	JVM 级别的严重问题,程序无法恢复	段错误、OOM kill	不处理(处理也没用)
Checked Exception	IOException、SQLException	可预料的业务/IO 失败,调用方应当处理	文件打开失败等"可恢复错误"	必须(声明或捕获)
RuntimeException(Unchecked)	NullPointerException、 IndexOutOfBoundsException、 IllegalArgumentException	编程错误,修复代码而不是捕获	空指针解引用、越界	不必(不强制)

★ 重点:处理还是不处理,这是个规则问题

Checked 异常:编译器强制你面对。要么方法 `throws IOException` 声明"我会甩锅给调用方",要么 try-catch 就地处理——二选一,漏了编译失败。Unchecked 异常:编译器不管,但你必须知道它不该被"处理"——NullPointerException 的正确修法是"判空",不是 catch 它。生产代码里 catch RuntimeException 几乎都是坏味道(第9.3节)。

§9.2 checked 与 unchecked:规则与取舍

示例 9-1 声明与处理的三条路

```

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;

public class ExceptionDemo {
    // 路 1:声明 throws,把异常甩给调用方(适合"我不负责恢复"的层)
    static String readFile(String path) throws IOException {
        return Files.readString(Path.of(path), StandardCharsets.UTF_8);
    }

    // 路 2:捕获处理,就地恢复(适合"我能处理"的层)
    static String readOrDefault(String path) {
        try {
            return Files.readString(Path.of(path), StandardCharsets.UTF_8);
        } catch (IOException e) {
            System.err.println("读文件失败,用默认配置: " + e.getMessage());
            return "{}"; // 降级方案
        }
    }

    // 路 3:捕获后包装再抛(换语义,加上下文)
    static String readConfig(String path) throws IllegalStateException {
        try {
            return Files.readString(Path.of(path), StandardCharsets.UTF_8);
        } catch (IOException e) {
            throw new IllegalStateException("配置文件读取失败: " + path, e);
            // 原异常作为 cause 保留:调用栈不丢
        }
    }

    public static void main(String[] args) {
        System.out.println(readOrDefault("missing.txt")); // 降级输出 {}
        // readFile 必须处理:要么 try-catch,要么 main 也 throws
        try {
            System.out.println(readFile("app.properties"));
        } catch (IOException e) {
            System.out.println("main 层兜底: " + e.getMessage());
        }
    }
}

```

⚠ 易错点 1:吞异常与过度声明

两个反面典型: ① `catch (Exception e) {}` 空手吞掉——错误无声消失,生产事故的温床;至少打日志,要么抛出去。 ② `main` 一杆子 `throws Exception`——程序入口的异常只被 JVM 打印堆栈,等于没处理。工程惯例:底层"抛"(throws),中间层"包"(wrap 加上下文),边界层"处理"(恢复或告警退出),入口绝不吞。

§9.3 try-catch-finally 与自定义异常

9.3.1 多 catch 与 finally

多 catch 顺序:子类在前,父类在后(先 `catch FileNotFoundException` 再 `catch IOException`,否则子类永远轮不到)。
`finally` 无论是否异常都执行,常用来清理——但第5章已说,资源清理优先 `try-with-resources`; `finally` 留给人"关闭手动资源、统计、解锁"。JDK7+ 支持多异常一个 catch: `catch (A | B e)`。

示例 9-2 多 catch 与 finally 执行顺序

```

public class FinallyDemo {
    public static void main(String[] args) {
        try {
            System.out.println("try 里");
            if (args.length > 0 && args[0].equals("boom")) {
                throw new IllegalArgumentException("故意出错");
            }
            return; // 提前 return,finally 照样执行!
        } catch (IllegalArgumentException e) {
            System.out.println("catch: " + e.getMessage());
        } catch (RuntimeException e) { // 父类在后
            System.out.println("catch 父类: " + e);
        } finally {
            System.out.println("finally:无论怎样都执行");
        }
        System.out.println("finally 之后:仅无异常/异常已处理才到");
    }
}
// 输出:try 里 → catch → finally;或:try 里 → finally → finally 之后

```

示例 9-3 自定义异常

```

// 业务异常:继承 RuntimeException(调用方不必强制处理,由框架统一捕获)
public class BusinessException extends RuntimeException {
    private final String code; // 业务错误码,给前端/日志用

    public BusinessException(String code, String message) {
        super(message);
        this.code = code;
    }

    public String getCode() { return code; }
}

// 用法示例
class OrderService {
    void pay(long orderId, double amount) {
        if (amount <= 0) {
            throw new BusinessException("INVALID_AMOUNT", "金额必须为正");
        }
        // 业务逻辑...
        throw new BusinessException("BALANCE_NOT_ENOUGH", "余额不足");
    }
}

```



工程建议:异常是 API 的一部分

生产团队会维护"异常码字典":自定义异常携带错误码与可读消息,日志与前端共用。三个纪律:① 自定义异常命名以 `Exception` 结尾;② 携带 `cause` 不丢根因;③ 校验失败用 `IllegalArgumentException/IllegalState`,不要发明新异常。业务错误(余额不足)与系统错误(数据库挂了)分开建模,这是大型系统的基础设施。

§9.4 线程:创建与生命周期

9.4.1 三种创建方式

方式	写法	适用
Thread 子类	<code>class T extends Thread { public void run() {...} }</code>	不推荐(继承占掉名额)
Runnable 接口	<code>new Thread(() -> {...}).start()</code>	推荐:任务与线程解耦
Callable + Future	<code>executor.submit(callable) → future.get()</code>	需要返回值/异常

示例 9-4 线程创建与生命周期

```
public class ThreadDemo {
    public static void main(String[] args) throws Exception {
        // 方式 1:Runnable + lambda,最常用
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                System.out.println("线程1: " + i);
                try { Thread.sleep(50); } // 让出 CPU,睡 50ms
                catch (InterruptedException e) { return; }
            }
        }, "worker-1"); // 线程名:日志排查关键
        t1.start(); // 启动(不是 run!)

        // 方式 2:Callable 拿返回值
        java.util.concurrent.ExecutorService pool =
            java.util.concurrent.Executors.newSingleThreadExecutor();
        java.util.concurrent.Future<Integer> f = pool.submit(() -> {
            int sum = 0;
            for (int i = 1; i <= 100; i++) sum += i;
            return sum; // 有返回值,异常也会带回
        });
        System.out.println("Callable 结果: " + f.get()); // 5050
        pool.shutdown(); // 线程池必须关

        t1.join(); // 等 t1 结束再退出 main
        System.out.println("main 结束");
    }
}
// 生命周期:NEW(start 前)→ RUNNABLE → BLOCKED/WAITING/TIMED_WAITING → TERMINATED
```

⚠ 易错点 2:t.start() 与 t.run()

start() 开新线程,run() 只是当前线程里调个方法。写成 t.run() 的后果:任务在 main 线程串行执行,完全没有并发,且不报错——最隐蔽的并发 bug 之一。另外线程的异常:run() 里抛出的 RuntimeException 不会被调用方捕获(线程边界隔离),生产用 UncaughtExceptionHandler 记录日志。

§9.5 同步:竞态、锁与原子性

9.5.1 竞态演示与修复

C 的 counter++ 在多线程下会丢更新;Java 的 counter++ 一样(读-改-写不是原子的)。修法三选一:synchronized 锁、原子类 AtomicInteger、或者干脆不共享。C 里是 pthread_mutex,Java 里是 synchronized 关键字。

示例 9-5 竞态:坏例子与三个修复

```
import java.util.concurrent.atomic.AtomicInteger;

public class RaceDemo {
    // 坏例子:普通 int,并发 ++ 会丢计数
    static int bad = 0;

    // 修复 1:原子类(无锁,最适合"计数器"这种单一操作)
    static final AtomicInteger good = new AtomicInteger();

    // 修复 2:synchronized 方法(锁住整个方法)
    static synchronized void incBad() { bad++; }

    public static void main(String[] args) throws Exception {
        int N = 10; // 10 个线程
        var pool = java.util.concurrent.Executors.newFixedThreadPool(N);
        for (int t = 0; t < N; t++) {
            pool.submit(() -> {
                for (int i = 0; i < 10000; i++) {
                    bad++; // 竞态:结果 ≤ 100000
                    good.incrementAndGet();// 原子:结果 = 100000
                }
            });
        }
        pool.shutdown();
        while (!pool.awaitTermination(10, java.util.concurrent.TimeUnit.SECONDS)) {
        }
        System.out.println("普通 int(可能有竞态): " + bad);
        System.out.println("AtomicInteger(必定正确): " + good.get());
    }
}
```

★ 重点:可见性 —— 比锁更隐蔽的问题

多线程下,一个线程改了变量,另一个线程 **不一定立刻看得到** ——CPU 缓存与编译器优化导致"副本"存在。synchronized 与 volatile 都保证可见性;普通字段不保证。volatile 适合"一写多读的开关标志"(如停止标志),但不解决"读-改-写"复合操作的原子性——计数还得原子类或锁。生产判断:不确定就用锁或原子类,volatile 是精打细算时的优化。

§9.6 java.util.concurrent:并发工具箱

9.6.1 线程池三件套

工具	用途	要点
Executors.newFixedThreadPool(n)	固定线程池	并发上限可控;任务排队
Executors.newCachedThreadPool()	弹性线程池	空闲 60s 回收;别接无界任务(会建海量线程)
Executors.newSingleThreadExecutor()	单线程串行	任务排队执行,天然串行化
Future.get()	取任务结果	阻塞等待;任务异常在 get 时抛出(ExecutionException)
shutdown()	优雅关闭	不再接受新任务,跑完存量;忘了关 = 进程不退出

9.6.2 并发容器

场景	C 的直觉	Java 正解
多线程读写 Map	HashMap + 手工锁	ConcurrentHashMap(分段锁/无锁,读无锁写有限竞争)
多线程读写 List	数组 + 锁	CopyOnWriteArrayList(读多写少)或 Collections.synchronizedList
生产者消费者队列	队列 + 条件变量	ArrayBlockingQueue / LinkedBlockingQueue(自带阻塞)
计数器	int + 锁	AtomicLong / LongAdder(高并发计数)

示例 9-6 线程池 + 并发容器:并发统计

```
import java.util.List;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.LongAdder;

public class ConcurrentDemo {
    public static void main(String[] args) throws Exception {
        // 场景:多个线程同时统计"各年级人数"
        List<String> grades = List.of("高一", "高二", "高三", "高一", "高一",
                                     "高三", "高二", "高一");

        ConcurrentHashMap<String, LongAdder> counts = new ConcurrentHashMap<>();
        ExecutorService pool = Executors.newFixedThreadPool(4);

        for (String g : grades) {
            pool.submit(() -> {
                // computeIfAbsent:没有就建 LongAdder,原子地做
                counts.computeIfAbsent(g, k -> new LongAdder()).increment();
            });
        }
        pool.shutdown();
        pool.awaitTermination(5, java.util.concurrent.TimeUnit.SECONDS);

        counts.forEach((k, v) -> System.out.println(k + ": " + v.sum()));
        // 高一: 4, 高二: 2, 高三: 2
    }
}
```

⚠ 易错点 3:HashMap 并发写会"挂"而不是"错"

多线程写普通 HashMap 可能死循环(JDK7)/丢数据(JDK8),而不是抛异常——这比"结果不对"危险得多,因为无声。凡是并发环境出现的共享 Map,默认 ConcurrentHashMap;JDK8 的 ConcurrentHashMap 读完全无锁,性能损失极小,没有理由不用。

§9.7 死锁与并发纪律

9.7.1 死锁四条件与工程对策

死锁四条件:C 里学的(互斥、占有且等待、不可剥夺、循环等待)在 Java 完全适用。工程对策:① 锁的顺序全局一致(都先锁 A 再锁 B);② 尽量用无锁方案(原子类、ConcurrentHashMap);③ 用 tryLock 加超时,拿不到就放弃而不是死等。

示例 9-7 死锁演示与 tryLock 修复

```

import java.util.concurrent.locks.ReentrantLock;

public class DeadlockDemo {
    static final Object A = new Object();
    static final Object B = new Object();

    // 死锁版:两个线程按相反顺序抢锁,互相等对方释放
    static void deadlock() {
        Thread t1 = new Thread(() -> {
            synchronized (A) {
                System.out.println("t1 拿到 A");
                try { Thread.sleep(50); } catch (InterruptedException e) { }
                synchronized (B) { System.out.println("t1 拿到 B"); } // 等 t2 放 B
            }
        });
        Thread t2 = new Thread(() -> {
            synchronized (B) {
                System.out.println("t2 拿到 B");
                try { Thread.sleep(50); } catch (InterruptedException e) { }
                synchronized (A) { System.out.println("t2 拿到 A"); } // 等 t1 放 A
            }
        });
        t1.start();
        t2.start();
        // 两个线程永远卡在"互相等":程序僵死,无输出"拿到第二个锁"
    }

    // 修复版:tryLock 带超时,拿不到就放弃重试,不死等
    static void fixed() {
        var lockA = new ReentrantLock();
        var lockB = new ReentrantLock();
        boolean ok = lockA.tryLock() && lockB.tryLock(); // 非阻塞尝试
        if (ok) {
            try {
                System.out.println("两个锁都拿到,干活");
            } finally {
                lockB.unlock();
                lockA.unlock(); // 释放顺序与加锁相反
            }
        } else {
            System.out.println("拿不到锁,放弃(或稍后重试)"); // 锁未全拿时别部分释放
        }
    }
}

```



工程建议:并发纪律十条(节选)

- 能不改共享状态就不改:局部变量优先,不可变对象优先 (第14章的 record 是利器)
- 共享可变状态:要么锁,要么原子类,要么并发容器;三选一,不许裸奔
- 锁的粒度:锁短代码块而不是整个方法;但别把原子操作拆碎(拆碎 = 新竞态)
- 线程名、日志、异常 cause 三件套齐全,否则线上问题无法定位
- 线程池一律 shutdown;写线程的代码先想"它什么时候退出"

本章速查表

主题	要点
异常体系	Throwable → Error(不处理)/ Exception(处理)/ RuntimeException(修代码)
checked 规则	要么 throws 声明,要么 try-catch;漏了编译失败;IOException 是典型
unchecked	NPE/越界/IllegalArgument;正确修法:判空/校验,不是 catch
多 catch	子类在前父类在后;JDK7+ 可 catch(A B e)
finally	永远执行(return 也执行);资源清理优先 try-with-resources
自定义异常	继承 RuntimeException + 错误码;命名 Exception 结尾;保留 cause
线程创建	Runnable + lambda 最常用;Callable 拿返回值;start() 不是 run()
生命周期	NEW → RUNNABLE → BLOCKED/WAITING/TIMED_WAITING → TERMINATED
竞态	++ 不是原子;修复:原子类 / synchronized / 不共享
可见性	volatile 保证可见不保证原子;不确定就用锁或原子类
线程池	Executors.newFixedThreadPool(n);任务 Runnable/Callable;必须 shutdown
并发容器	ConcurrentHashMap / CopyOnWriteArrayList / BlockingQueue / LongAdder
死锁	锁顺序一致 / tryLock 超时 / 无锁优先;四条件复现诊断

最小必会清单

- 能画出异常体系树并解释三种类型的处理策略
- 能说出 checked 异常的两条合法出路,能举出三种典型
- 能写出多 catch + finally,并说出 finally 在 return 时仍执行
- 能用 lambda 创建线程并说出 start/run 的区别
- 能解释竞态与可见性,写出 AtomicInteger 计数
- 能说出三种线程池的适用场景与 shutdown 的必要性
- 能解释死锁四条件并写出 tryLock 修复

自测题

Q 1. 下面的异常各属哪类,该怎么处理?① 文件不存在;② 数组越界;③ 内存耗尽;④ 余额不足(业务)

✅ 参考答案

① IOException,checked:声明或处理(降级/提示)。② IndexOutOfBoundsException,unchecked:修代码(检查下标),不是 catch。③ OutOfMemoryError,Error:不捕获,优化内存使用(第5章)。④ 自定义业务异常(继承 RuntimeException):由框架统一捕获返回错误码,业务层不散落 try-catch。

Q 2. 为什么说"catch (Exception e) { }"是坏味道?至少写出两种正确做法。

✅ 参考答案

空 catch 吞掉错误:程序继续跑,但状态可能已坏,问题被掩盖到无法排查。正确做法:① 至少打日志(log.error("...", e));② 抛回或包装(throw new BusinessException(...));③ 能恢复就显式降级(返回默认值并记录)。原则:异常要么处理,要么传递,不许消失。

Q 3. 两个线程对同一个 int 做 10 万次 ++,结果可能小于 20 万。为什么?给出两种修复。

✓ 参考答案

++ 是"读-加-写"三步,两个线程可能同时读到同一个值,各自 +1 再写回,一次更新被覆盖(丢失更新)。修复:AtomicInteger.incrementAndGet()(无锁 CAS);或 synchronized 方法/块(锁)。注意 volatile 不能修复:它管可见性,不管复合操作原子性。

Q 4. Future.get() 会抛哪些异常?任务里抛的 RuntimeException 怎么拿到?

✓ 参考答案

InterruptedException(被中断)与 ExecutionException(任务抛出的异常被包装)。任务内的 RuntimeException 会作为 ExecutionException 的 cause 抛出,用 e.getCause() 拿到原始异常。所以"线程里的异常无人知"只发生在裸 Thread 的 run() 里;用线程池 + Callable 异常能被捕获。

Q 5. 生产环境 JVM 进程不退出,排查第一步看什么?为什么?

✓ 参考答案

jstack 打印线程栈,找"卡住的线程":最常见的是非 daemon 线程池没 shutdown(线程池线程不会自己退出),或死锁(栈里能看到两个线程互相持有对方等待的锁)。C 的 gdb attach 是同类操作。看完线程再看资源:jcmd Thread.print 同样有效。

Q 6. ConcurrentHashMap 的 computeIfAbsent 是干什么的?为什么比"先 get 再 put"安全?

✓ 参考答案

computeIfAbsent(key, fn):键不存在时原子地执行 fn 生成值并放入,存在则返回已有值。"先 get,不存在再 put"两步在并发下会产生竞态:两个线程同时发现不存在,各自 put,后写的覆盖先写的(丢失更新)。computeIfAbsent 把"检查-创建-放入"合成一个原子操作。

章末实验题:并发成绩统计系统

实验 9:并发成绩统计系统

实验目标:编写一个多线程成绩统计系统,综合运用本章全部知识点:异常体系、checked 处理、自定义异常、线程、竞态修复、原子类、线程池、并发容器。

实验要求:

- 任务 1:定义 GradeFileException extends RuntimeException 携带错误码(覆盖 §9.1-9.3 异常)
- 任务 2:读取 scores.csv(可缺失),文件不存在时抛 GradeFileException("FILE_NOT_FOUND"),main 捕获后打印友好提示(覆盖 §9.2-9.3 checked 处理)
- 任务 3:每条记录解析时校验格式,非法行跳过并计数(覆盖 §9.3 多 catch 与防御)
- 任务 4:用 4 线程线程池并行统计"每个分数段的人数"(0-59/60-79/80-100),每个线程处理一批数据(覆盖 §9.4-9.5 线程与竞态)
- 任务 5:统计计数器用 LongAdder 与 ConcurrentHashMap,保证并发正确(覆盖 §9.5-9.6 原子与并发容器)
- 任务 6:定义 volatile 停止标志,主线程统计完成后让工作线程优雅退出(覆盖 §9.5 可见性)
- 任务 7:main 汇总输出:总人数、各分数段人数、非法行数、耗时(覆盖全部)

实验环境:JDK 17+;scores.csv 至少 200 行数据(可手工生成,或运行任务 8:内置生成器)。

第一步 写串行版(读文件、解析、统计),验证正确,再加线程

第二步 务 4 的分批:把总行数均分成 4 段,每线程处理一段;注意"下标边界"

第三步 务 6 的停止标志:工作线程循环里检查 flag,主线程置位后 join

✅ 参考答案要点

```

import java.io.BufferedReader;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.LongAdder;
import java.util.concurrent.atomic.AtomicInteger;

/** 并发成绩统计系统 — 第9章综合实验参考答案
 * 覆盖:异常体系、自定义异常、多 catch、线程池、原子类、并发容器、volatile
 */
public class ConcurrentScoreStats {

    // 任务 1:自定义业务异常,携带错误码
    static class GradeFileException extends RuntimeException {
        private final String code;

        GradeFileException(String code, String message) {
            super(message);
            this.code = code;
        }

        String getCode() { return code; }
    }

    public static void main(String[] args) {
        try {
            List<Integer> scores = loadScores(Path.of("scores.csv"));
            report(scores);
        } catch (GradeFileException e) {
            System.out.println("业务错误[" + e.getCode() + "]: " + e.getMessage());
        } catch (Exception e) { // 兜底:防止线程边界把异常吞掉
            System.out.println("系统错误: " + e);
        }
    }

    // 任务 2 + 3:读取与校验;文件缺失抛业务异常
    private static List<Integer> loadScores(Path path) {
        List<Integer> scores = new ArrayList<>();
        if (!Files.exists(path)) {
            throw new GradeFileException("FILE_NOT_FOUND", "找不到 " + path);
        }
        try (var reader = Files.newBufferedReader(path, StandardCharsets.UTF_8)) {
            String line;
            int lineNo = 0;
            while ((line = reader.readLine()) != null) {
                lineNo++;
                try {
                    int s = Integer.parseInt(line.trim());
                    if (s < 0 || s > 100) {
                        System.out.println("第 " + lineNo + " 行分数越界,跳过: " + line);
                        continue;
                    }
                    scores.add(s);
                } catch (NumberFormatException e) {
                    System.out.println("第 " + lineNo + " 行格式非法,跳过: " + line);
                }
            }
        } catch (java.io.IOException e) { // checked:包装成业务异常向上抛
            throw new GradeFileException("IO_ERROR", "读取失败: " + e.getMessage());
        }
        return scores;
    }
}

```

```

// 任务 4-7:并发统计
private static void report(List<Integer> scores) throws Exception {
    // 分数段表:并发安全的"段名 → 计数器"
    ConcurrentHashMap<String, LongAdder> buckets = new ConcurrentHashMap<>();
    // 任务 6:volatile 停止标志(一写多读)
    // 用 AtomicBoolean 替代,效果等同且类型安全
    AtomicInteger working = new AtomicInteger(scores.size());

    int threads = 4;
    ExecutorService pool = Executors.newFixedThreadPool(threads);
    int chunk = (scores.size() + threads - 1) / threads;

    long start = System.currentTimeMillis();
    for (int t = 0; t < threads; t++) {
        final int from = t * chunk;
        final int to = Math.min(from + chunk, scores.size());
        pool.submit(() -> {
            for (int i = from; i < to; i++) {
                int s = scores.get(i);
                String bucket;
                if (s < 60) bucket = "0-59";
                else if (s < 80) bucket = "60-79";
                else bucket = "80-100";
                // computeIfAbsent:并发安全地取/建 LongAdder
                buckets.computeIfAbsent(bucket, k -> new LongAdder()).increment();
                working.decrementAndGet(); // 进度计数(原子)
            }
        });
    }
    pool.shutdown(); // 优雅关闭:不再接新任务
    boolean done = pool.awaitTermination(30, TimeUnit.SECONDS);
    if (!done) {
        System.out.println("警告:统计超时未完成");
    }
    long cost = System.currentTimeMillis() - start;

    // 任务 7:汇总报告
    System.out.println("== 并发成绩统计报告 ==");
    System.out.println("总人数: " + scores.size());
    buckets.forEach((k, v) -> System.out.println("  " + k + " 分: "
        + v.sum() + " 人"));
    System.out.println("处理完成标记: " + (working.get() == 0));
    System.out.println("耗时: " + cost + " ms(4 线程)");
}
}

```

设计说明:任务 2 故意让 loadScores 抛"业务异常"而非裸 IOException——数据文件缺失是业务事件,应由调用方决定文案;任务 5 用 computeIfAbsent 保证"建桶 + 计数"原子,这正是自测题 6 的实战版;任务 6 用 AtomicInteger 当"剩余工作量"计数器,既验证了所有任务都执行完,也演示了原子类的进度追踪(volatile 只能做布尔开关,计数必须原子)。扩展方向:改成 Callable 让每个线程返回自己的统计,再合并(Future.get 的错误会暴露在主线);加大数据量(100 万行)对比串行与并行的耗时。

下一章预告:第10章 泛型与集合框架。ArrayList 与 LinkedList、HashMap 原理、泛型与类型擦除、PECS 原则——集合是并发之后的下一块基石。